

1. COS 226, f25, precept 1. Tìm bug

Hãy xem cài đặt không chính xác sau đây của hàm `union()` trong cấu trúc dữ liệu quick-find. Lưu ý rằng mảng `leader[]` có độ dài n được khởi tạo với `leader[i] = i` cho mọi i , và hàm `find(i)` trả về giá trị `leader[i]`.

```
1 public void union(int p, int q) {
2     for (int i = 0; i < leader.length; i++)
3         if (leader[i] == leader[p])
4             leader[i] = leader[q];
5 }
```

Hãy một giá trị cho số lượng phần tử n , một chuỗi các phép toán `union()`, một số nguyên $0 \leq i < n$ và một số nguyên $0 \leq j < n$, sao cho các phần tử i và j thuộc cùng một tập hợp nhưng `find(i)` và `find(j)` trả về các giá trị khác nhau. Nói cách khác là tìm 1 testcase mà cài đặt trên chạy sai.

2. COS226, Midterm f22

Giả sử ta khởi tạo một cấu trúc dữ liệu union-find với n phần tử. Sau đó, ta thực hiện chuỗi các thao tác `union()` sau: `union(0, 1)`, `union(0, 2)`, `union(0, 3)`, ..., `union(0, n-1)`.

Trả lời các câu hỏi sau:

(a) Cấu trúc dữ liệu thu được chứa tổng cộng bao nhiêu thành phần liên thông (tức là các tập hợp rời nhau)?

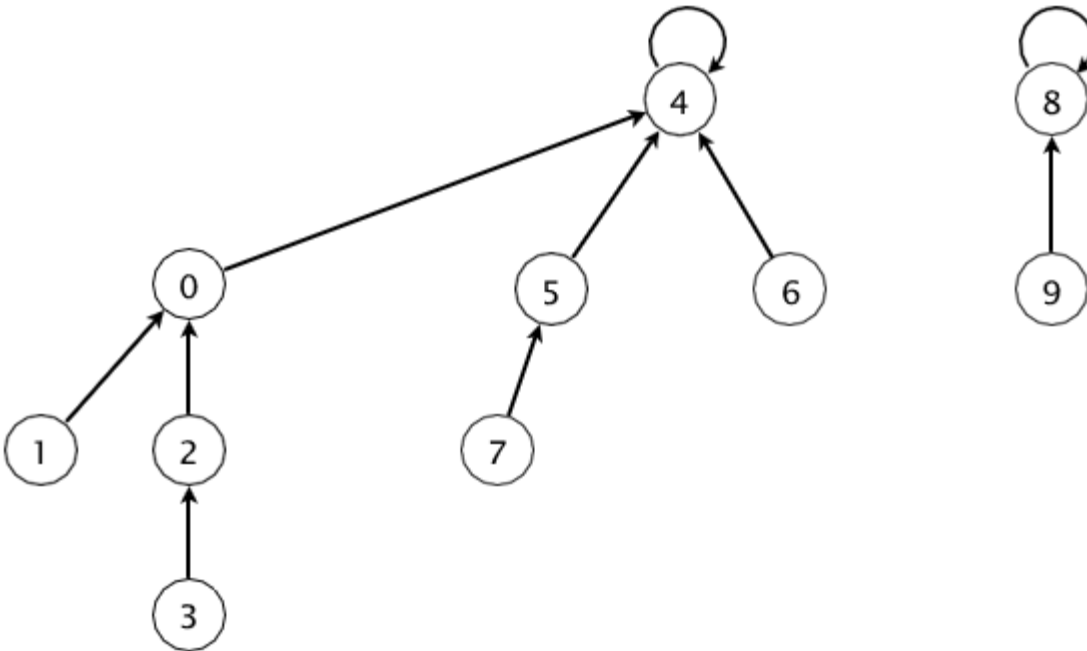
(b) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp quick-find. Có bao nhiêu lần cập nhật mảng được thực hiện bởi các thao tác `union()` này (biểu diễn dưới dạng hàm của n theo ký hiệu xấp xỉ $\sim$)? (Lưu ý: cài đặt quick-find cho `union(p, q)` của chúng ta không bao giờ thay đổi giá trị `leader[q]`.)

(c) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp quick-union, và ta gọi `find(0)` sau chuỗi thao tác nêu trên. Thao tác `find(0)` sẽ thực hiện bao nhiêu lần truy cập mảng (biểu diễn dưới dạng hàm của n theo ký hiệu Θ)? (Lưu ý: cài đặt quick-union cho `union(p, q)` của chúng ta không bao giờ thay đổi `parent[q]`.)

(d) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp weighted quick-union, và ta gọi `find(0)` sau chuỗi thao tác nêu trên. Thao tác `find(0)` sẽ thực hiện bao nhiêu lần truy cập mảng (biểu diễn dưới dạng hàm của n theo ký hiệu Θ)? (Lưu ý: cài đặt weighted quick-union cho `union(p, q)` của chúng ta sẽ thay đổi `parent[q]` nếu hai cây có cùng kích thước.)

3. COS226, Midterm f25, 2a

Xét biểu diễn *parent-link* dưới đây của một cấu trúc dữ liệu *weighted quick union* (*link-by-size*):



Trong các cặp (p, q) dưới đây, những cặp nào có thể là tham số của lệnh $\text{union}(p, q)$ gần nhất? Giả thiết rằng p và q thuộc hai tập khác nhau trước khi lệnh $\text{union}(p, q)$ được chạy.

$(0, 4)$ $(2, 3)$ $(4, 0)$ $(4, 5)$
 $(5, 0)$ $(5, 7)$ $(6, 4)$ $(8, 9)$

chú ý: khi gọi $\text{union}(p, q)$ mà cây chứa p và cây chứa q có kích thước bằng nhau thì ta sẽ trở nút gốc cây chứa q tới gốc của cây chứa p .

4. COS226, Midterm s25, 2a

Xét biểu diễn *parent-link* dưới đây của một cấu trúc dữ liệu *weighted quick union* (*link-by-size*):

parent[]	0	0	0	0	4	4	4	5	?	8
	0	1	2	3	4	5	6	7	8	9

$\text{parent}[8]$ có thể nhận những giá trị nào trong các giá trị 0, 1, ..., 9

5. COS226, Midterm f24, 4

Dưới đây có thể là biểu diễn *parent-link* của một cấu trúc dữ liệu *weighted quick union* hay không?

0	1	2	3	4	5	6	7	8	9
5	6	0	4	0	5	0	5	6	7